

Beispiele CAT – Steuerung



- 1. ICOM-705 als Remote-Station
 - Steuerung über RaspberryPi
- 2. YAESU FT-710 mit externem Tuner
 - 2.1 Steuerung über PC
 - 2.2 Steuerung über ESP32
- Verweise

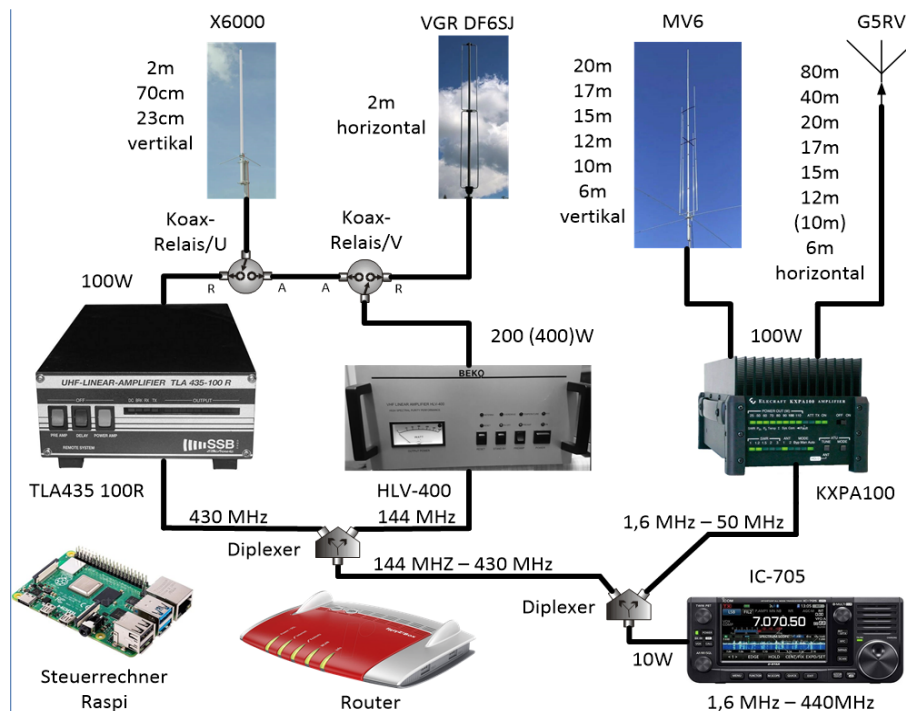
dk2jk 24.09.2026 Treff Darc

Dieses Projekt zeigt an 2 Beispielen die Steuerung von Amateurfunkgeräten über die CAT-Schnittstelle (Computer Aided Transceiver).

Die Skripte wurden auf einem Linux-Laptop entwickelt und für den Einsatz auf einen Raspberry Pi sowie einen ESP32 portiert.

Die Python-Skripte sind klein gehalten, um die reine CAT-Kommunikation verständlich zu zeigen.

1. Remote- Station mit IC-705



24.9.26 DK2JK 2/25

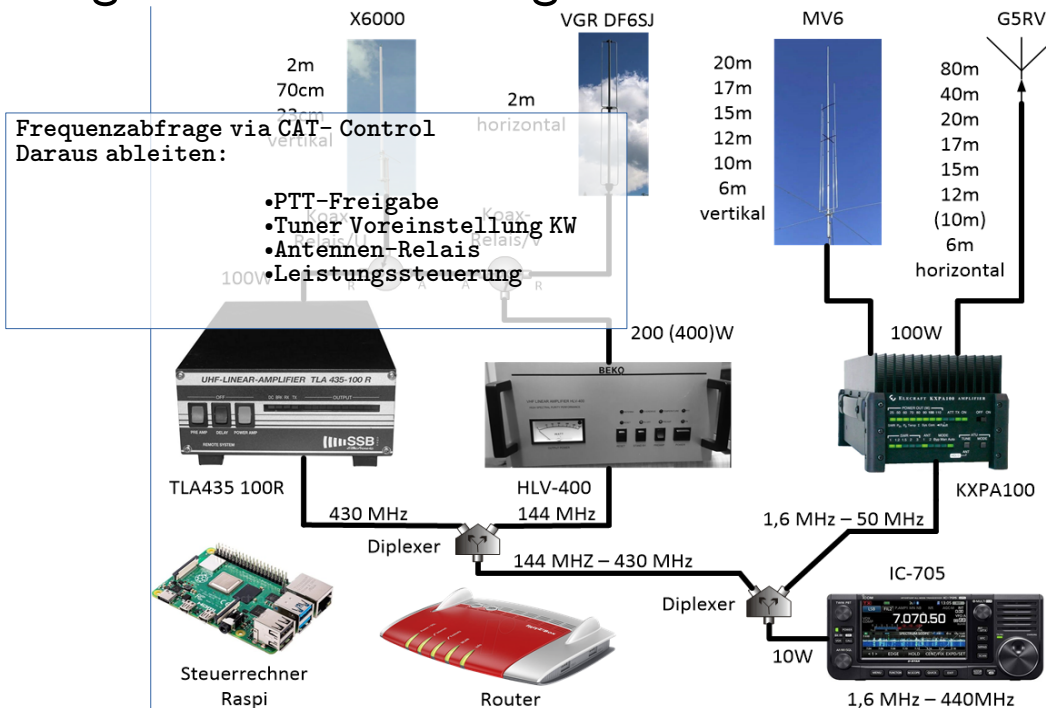
Beim ersten Beispiel handelt es sich um eine Remote-Station mit einem IC-705; zur Ausrüstung gehört noch je eine Endstufe für 70cm, 2m und KW. Die Antennen werden automatisch oder ferngesteuert gewählt.

Der Transceiver wird über die Remote- Software WFVIEW ferngesteuert.

Über WFVIEW ist jedoch die Zuschaltung individueller Endstufen und die Umschaltung der Antennen nicht möglich.

Hier kommt ein RaspberryPi ins Spiel, der die Frequenz abfragt und davon abhängig über ein Interface Steuersignale für Antennenrelais und Endstufen ausgibt.

Aufgabe der Steuerung



24.9.26 DK2JK 3/25

Die eingestellte Frequenz bestimmt fast alles!

Abhängig von der Frequenz wird der PTT-Eingang der jeweiligen Endstufen freigegeben.

Der Tuner der Elecraft KXPA100 wird mit der aktuellen Frequenz voreingestellt, so dass der Tuner auch bei Empfang optimal arbeitet.

WEB-Interface „Dashboard“

RRC IC-705

* Geräte Status *

Frequenz : 14.015 MHz
Aktive PA : keine
Antenne : KW: G5RV/MV6

Info:
KXPA: SWR ---
ICOM: Kommunikation OK

* Optionen *

Antennen-Polarisation (UHF/VHF)
☒ vertikal ☐ horizontal

PA freigeben
☐ ja ☒ nein

-Bei 2m vertical wird die Leistung
des Ic-705 auf 50% reduziert
-Auf 40m ist IMMER ANT1 (G5RV) gewählt.

Time [UTC]: 12.09.26 11:04:15
System Reboot @ 04:00 h
Version: 20.Jan.26

24.9.26 DK2JK 4/25

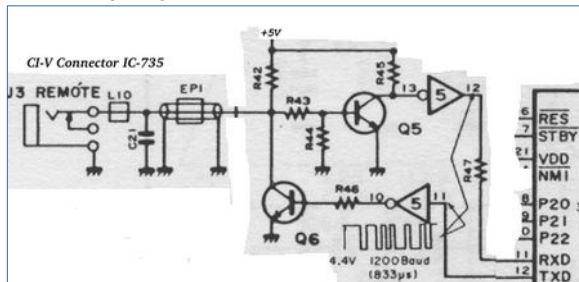
Weitere Optionen, wie PA An- oder Aus-Schalten oder Antennenwechsel sind über ein zusätzliches WEB-Interface einstellbar:

Das WEB-Interface ist aber hier kein Thema.

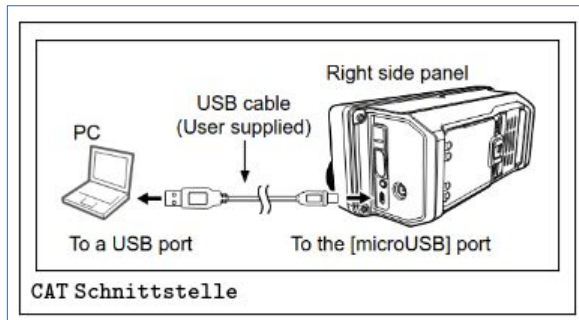
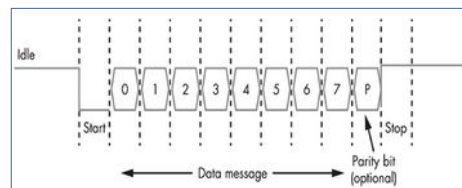
Beschäftigen wir uns nun mit der CAT-Steuerung des IC-705 ...

Hardware ICOM- CI-V (TTL)

CAT Steuerung = „Computer Aided Transceiver“



Datenformat „TTL-UART“



CI-V „Computer Interface V (Fünf)
TTL -Level (Ruhezustand ca +5V,
Aktivzustand ca. 0V)
Die Sende- und Empfangsleitung
(RXD und TXD) wird auf eine Leitung
kombiniert (Bus) .

**Beim IC-705 ist stattdessen eine
virtuelle serielle Schnittstelle
als USB-Schnittstelle vorhanden.**

24.9.26 DK2JK 5/25

So sieht zum Beispiel die CAT- Hardware bei einem IC-735 aus .

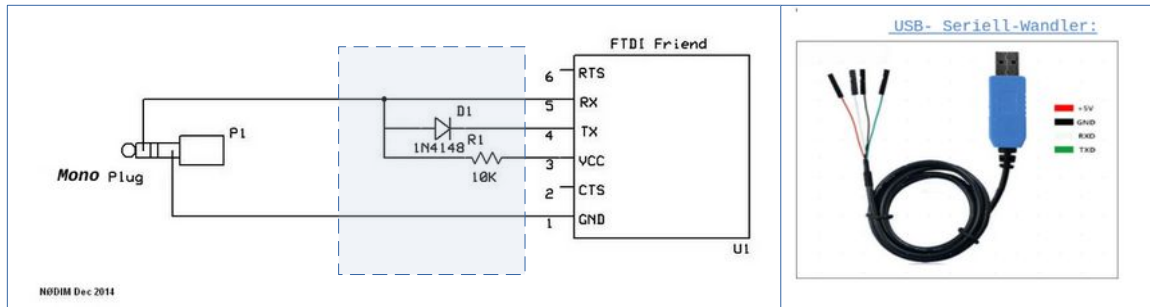
Grundsätzlich ist die CAT-Schnittstelle eine asynchrone serielle Schnittstelle mit Startbit , 8-Bit Daten, Stopbit. Die Baudrate wird im Transceiver eingestellt.

Bei ICOM heißt die Schnittstelle CI-V(fünf) . Ursprünglich ist es ein ‚Ein-Draht-Bus‘ mit TTL- Pegel. Es kann abwechselnd auf der Signalleitung gesendet oder empfangen werden; nur der Sender (TXD) zieht den Bus auf LOW. Das Schaltbild ist aus einem IC-735. (Anmerkung: da RXD und TXD die gleiche Signalleitung ist, hört der Sender sein TXD-Signal auf RXD mit.)

Beim IC-705 ist stattdessen eine virtuelle serielle Schnittstelle als USB-Schnittstelle vorhanden. Das ist bei den meisten modernen Transceivern der Fall. Eine TTL-Schnittstelle ist dann oft noch auf einer extra Buchse vorhanden .

CI-V – USB-Adapter selbstgebaut

z.B. für IC-706
(bei IC-705 nicht erforderlich)



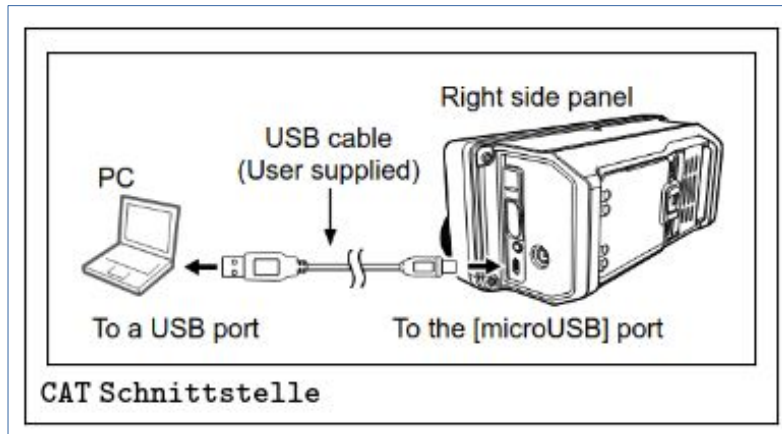
Adapter	---	CI-V Adapter
GND ,schwarz	---	G
TXD ,grün	---	rx
RXD ,weiß	---	tx
+5V	---	:

24.9.26 DK2JK 6/25

Für den Fall, dass eine USB-Schnittstelle benötigt wird, aber nur eine TTL- Schnittstelle vorhanden ist, so kann man mit einen USB- TTL- Adapter und ein wenig zusätzlicher Hardware einen USB-Adapter bauen.

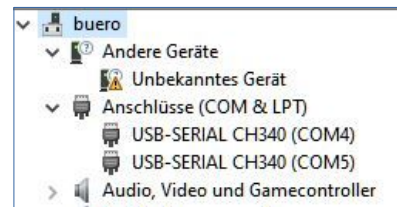
Hintergrund: Auf diese Weise konnte ich einen IC-706 über die USB-Schnittstelle steuern, da mir zu Beginn des Projekts kein IC-705 zur Verfügung stand.

USB als virtueller COM-Port



```
pi@raspi62:~/app $ ls /dev/ttyA*  
/dev/ttyACM0 /dev/ttyACM1  
pi@raspi62:~/app $ ls /dev/ttyU*  
/dev/ttyUSB0
```

24.9.26 DK2JK 7/25



Wenn man auf eine ser. Schnittstelle zugreifen will, stellt sich die Frage: „Wie heißt die Schnittstelle?“.

Da ein RaspberryPi mit Linux als Betriebssystem läuft, schaue ich im Verzeichnis „/dev/“ nach Geräten mit dem Namen „/dev/ttyACM0“ (das ist der IC-705) und nach Geräten mit dem Namen „/dev/ttyUSB0“ (das ist die KXPA100). Bei Linux wird mit dem Befehl „ls“ ein Verzeichnis gelistet.

Auf einem Windows-PC sucht man die Schnittstellen-Namen im Gerätemanager unter „Anschlüsse(COM)“; der Gerätemanager kann in der Kommandozeile mit „devmgmt.msc“ gestartet werden.

Die im folgenden vorgestellten Python-Programme für den Raspi laufen fast unverändert auf einem Linux- oder Windows-PC. Programmteile wurden am Laptop ausprobiert und später auf einen Raspi übertragen.

Ausschnitt IC-705-Command Table

◇ Command table

Aus:
ICOM CI-V REFERENCE GUIDE (English)

Cmd.	Sub cmd.	Data	Description
00		See p. 16.	Send the frequency data (transceive)
01		See p. 16.	Send the mode data (transceive)
02*1		See p. 16.	Read the band edge frequencies
03*1		See p. 16.	<u>Read the operating frequency</u>
04*1		See p. 16.	Read the operating mode
05*2		See p. 16.	Set the operating frequency
06*2		See p. 16.	Set the operating mode
07			Select the VFO mode

24.9.26 DK2JK 8/25

Wie komme ich an die Frequenz ?

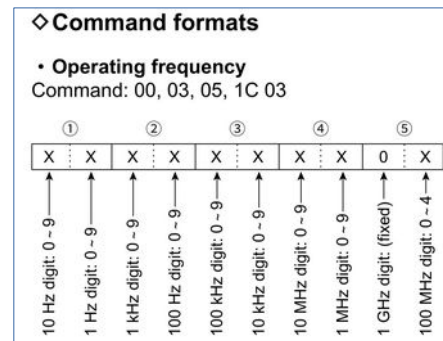
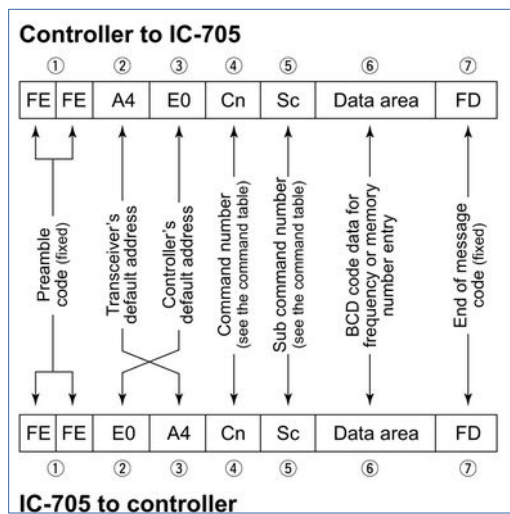
In der Kommandotabelle des IC-705
(ICOM CI-V REFERENCE GUIDE) suche ich nach dem
Kommando für die Frequenzabfrage.

Hier findet man ,03' für „read the operating
frequency“ , zu deutsch „aktuelle Frequenz
lesen“.

Weitere Kommando für unsere Anwendung benötigt
man zunächst nicht.

Für die Programmierung muss dieses Kommando in
einen Datenrahmen eingebaut werden.

CI-V Command Format ICOM-705



aus:
ICOM CI-V REFERENCE GUIDE (English)

So sieht das komplette CI-V - Datenpaket aus.

Bei ICOM werden die Daten hexadezimal dargestellt;
ein Hex- Buchstaben-Paar entspricht einem Byte.

Die mit dem Hexwert verbundenen Werte als Zahl sind unwichtig; wir betrachten nur die HEX-Codes.

Als nächstes wird die Codierung im Einzelnen am Beispiel der Frequenzabfrage erläutert...

Datenformat der Frequenzabfrage im Detail

```
#ICOM Kommando zum Frequenz lesen:
# FE FE A4 E0 03 FD
# FE FE          <- kopf
# ..... A4      <- IC-705 CAT Adresse
# ..... E0      <- default ??
# ..... 03      <- Kommando Frequenz lesen
# ..... FD      <- Ende Zeichen
```

Interpretation der Antwort:

```
Antwort      = FE FE E0 A4 03 00 50 43 28 00 FD
Nutzdaten    = 00 50 43 28 00
Nutzdaten    = 00 28 43 50 00 (gedreht)
Roh-Ergebnis = "0028435000" = text
Frequenz     = 28.435 MHz = float(text)/1e6
```

24.9.26 DK2JK 10/25

Das Datenformat CI-V besteht aus:

Präambel	FE FE	
CAT-Adresse	A4	(für IC-705)
Controller Adresse	E0	(ich)
Kommando	03	(Frequenz lesen)
Ende-Zeichen	FD	

Die Antwort besteht aus der Wiederholung der Daten des Kommandos, wobei Sender und Empfänger vertauscht sind ; dann folgen 5 Bytes Nutzdaten. Abgeschlossen wird das Protokoll mit dem Hexwert 0xFD.

Wenn man die Nutzdaten von hinten liest, so kann man an den HEX-Bytes schon die Frequenz lesen; das Ergebnis muss noch durch 1.000.000 geteilt werden, so erhält man die Frequenz in MHz.

Python Skript Frequenzabfrage ICOM-705

```
[ ic705_cat_mit kommentar.py ]
1
2 from serial import Serial      # modul serial laden
3
4                                # Kommentar mit Beispieldaten...
5 def read_frequenz_IC_705():    # Frequenz lesen ...
6     kommando = [0xFE, 0xFE, 0xA4, 0xE0]+ [0x03] + [0xFD]
7                                # FE FE A4 E0 03 FD
8     com = Serial('/dev/ttyACM0', 9600) # schnittstelle öffnen
9     com.write( bytes(kommando) )      # kommando schreiben
10    antwort = com.read_until(bytes([0xFD])) # FE FE E0 A4 03 00 50 12 07 00 FD
11    nutzdaten = antwort[5:9]           #
12    nutzdaten_rueckwaerts = nutzdaten[::-1] # (gedreht) 00 50 12 07 00
13    text = nutzdaten_rueckwaerts.hex()  # text = "0007125000"
14    mhz = float(text)/1000_000          # mhz = float("0007125000")/1e6
15    com.close()                       # schnittstelle schliessen
16    return mhz                        # ergebnis = 7.125
17
18 if __name__ == '__main__':
19     frequenz = read_frequenz_IC_705() # frequenz lesen
20     print ( f' f={frequenz} MHz' )    # und anzeigen: f=7.125 MHz

Shell
>>> %Run 'ic705_cat_mit kommentar.py'
f=7.125 MHz
```

24.9.26 DK2JK 11/25

Mit diesem Python Skript wird die Frequenz gelesen.

Zeile 19: die Funktion ,read_frequenz_IC705' wird liefert die Frequenz .

Im Einzelnen...

Zeile 6: das Kommando wird zusammengesetzt(eine Liste mit Werten, hier in hex eingegeben.)

Zeile 8: Schnittstelle wird eröffnet; der Name der Schnittstelle wurde vorher im Gerätemanager abgelesen.

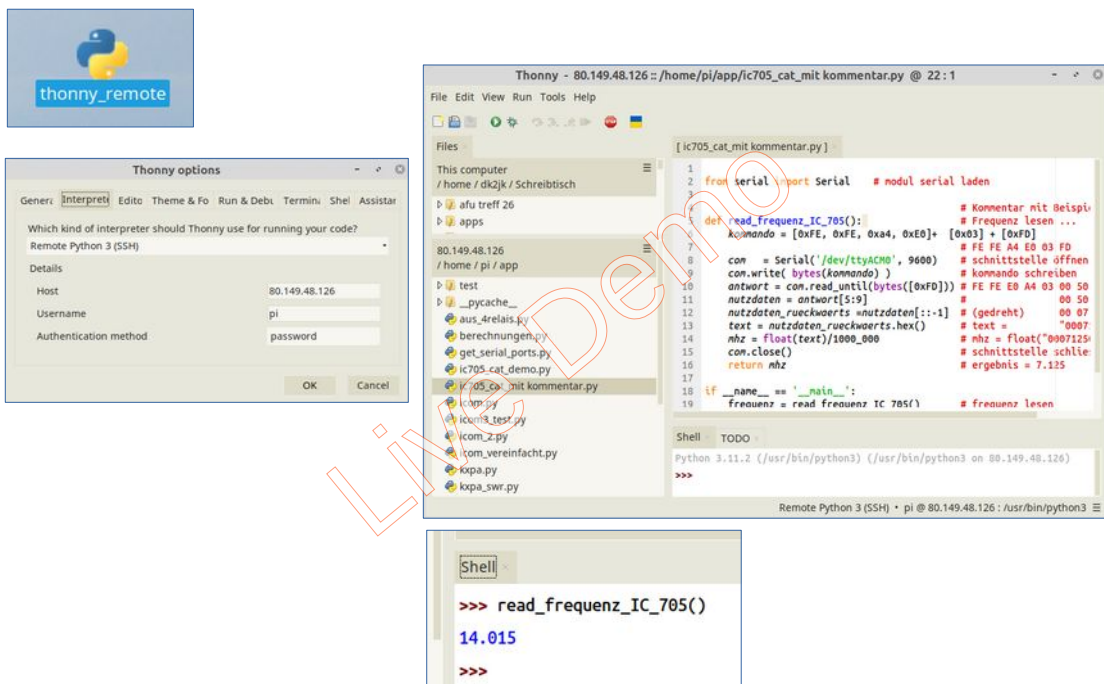
Zeile 9: Daten werden als Bytes geschrieben.

Zeile 10: Antwort wird gelesen (hier andere Frequenz als im vorherigen Beispiel)

Zeile 11 ...13: Nutzdaten werden herausgefiltert und formatiert

Zeile 14: Ergebnis in MHz

Python Skript interaktiv



24.9.26 DK2JK 12/25

Bei dieser Live-Vorführung wird das Programm THONNY verwendet, eine Python IDE mit Dateimanager, Editor und Shell.

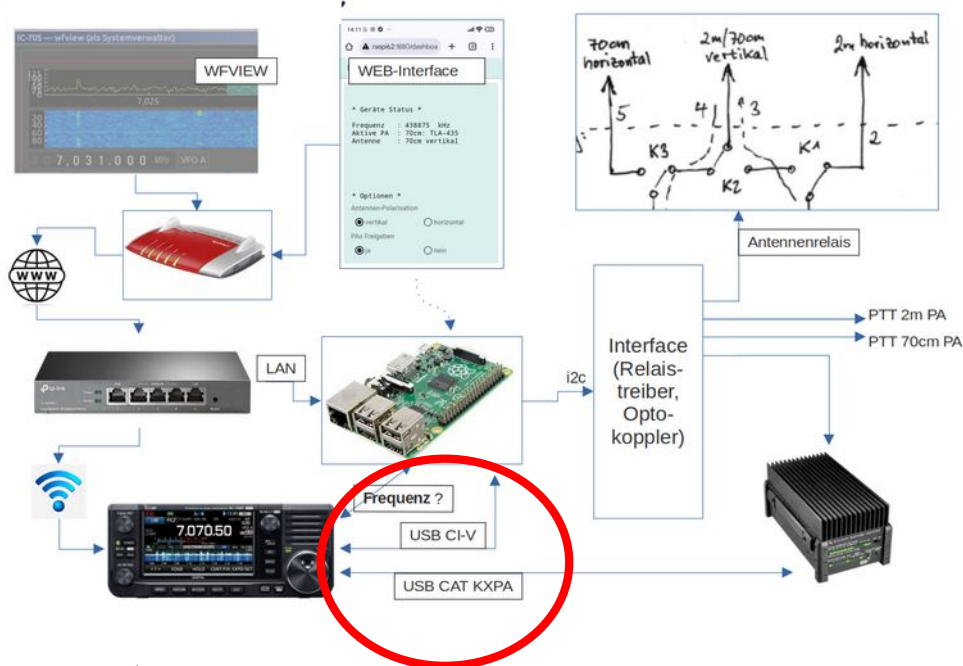
Thonny IDE wird hier fernbedient. Die Verbindung geht über SSH via Netzwerk mit der Ziel-Adresse „Host“.

Thonny läuft dann auf meinem Laptop, die Python Skripts laufen aber auf dem fernen Raspi.

Demonstration:

Aufruf der Funktion „read_frequenz_IC_705()“
interaktiv auf der Kommandozeile der Shell.

Wie hängt alles zusammen?



24.9.26 DK2JK 13/25

Hier ist nochmals die ganze Hardware der Remote-Station schematisch abgebildet.

Der RaspberryPi liest die Frequenz über die CAT-Schnittstelle.

Abhängig von der eingestellten Frequenz werden über ein I2C-Interface die passenden Antennen-Relais geschaltet. Außerdem wird die PTT der zugehörigen PA freigegeben. Der Tuner in der KXPA100 erhält ebenfalls die Frequenz-Info, damit der Tuner auch bei Empfang schon richtig steht.

Über ein zusätzlichen WEB-Interface ("Dashboard") werden die PAs ein- oder aus-geschaltet. Bei UKW kann von vertikaler auf horizontale Polarisierung umgeschaltet werden.

Das WEB-Interface wird hier nicht näher betrachtet (Stichworte hierzu : NodeRED, MQTT)

CAT-Control KXPA100

Der Tuner wird auf die Frequenz geschaltet noch bevor er überhaupt HF sieht.

Befehl: ^MT7032;
== Memory Tune 7032 kHz

Befehlsformat

KXPA100-Befehle beginnen mit einem Caret (^).
Alle Befehle verwenden den druckbaren ASCII-Zeichensatz.
Jeder serielle Befehl wird mit einem Semikolon (;)
abgeschlossen .

24.9.26 DK2JK 14/25

Hier eine kurze Info , wie ein CAT- Befehl bei der KXPA100 aussieht. Die Frequenz wird in kHz eingestellt.

Nebenbei:

Die CAT-Befehle des K2, K3 oder KX3 sehen ähnlich aus (Ohne (^)).

Da die Befehle in ASCII codiert sind, kann man sie ohne Umwandlung als Text lesen.

Bei Kennwood und YAESU sind die Befehle ebenfalls in ASCII codiert.

[pause]

2. Tune-Button für ext. Tuner am FT-710

Aufgabe: Ein externer Tuner soll zum Abstimmen mit einem 10W- Träger angesteuert werden.

Möglichkeiten:

1. Manuell:
Mode=AM (ca. 25W) --> PTT am Mikrofon drücken --> Tuner Arbeitet --> zurück auf alten ,Mode'
2. Verwendung der Buchse „REM/ALC“
Signal TX_REQ auf LOW --> TX sendet Träger
Leistung reduzieren mit negativen Spannung am Signal ALC; -4.9V --> 10W. (TUN/LIN PORT SELECT auf ,Linear')
3. CAT-Schnittstelle:
eine Kommandofolge senden, die den manuellen Tune-Vorgang nachbildet.

24.9.26 DK2JK 15/25

In einem zweiten Beispiel wird ein externer Tuner "SG-230" zum Abstimmen mit einem 10W -Träger angesteuert.

Es gibt 3 Möglichkeiten um einen Träger zum Abstimmen des Tuners zu senden.

1. Manuell: das ist mir zu umständlich

2. ALC: Funktioniert, man benötigt einen 3,5 mm Mono Stecker, einen Taster und eine negative Spannung, z.B. ein 9V Batterie plus Poti.

3. CAT: diese Schnittstelle habe ich mir zur Aufgabe gemacht.

Python Programm Frequenzabfrage YAESU

(Zum Vergleich: gleiche Aufgabe wie bei ICOM)

```
cat_pc_basic.py <
12
13 port = '/dev/ttyUSB0'      # hier anpassen
14 baud = 38400
15
16 my_ser = serial.Serial( port, baud )
17
18 def cat( cmd ):
19     my_ser.write(cmd.encode()) # ascii in bytes umwandeln und senden
20     time.sleep(0.2)           # etwas warten
21     antwort = my_ser.read_all() # und antwort lesen
22     return antwort.decode()   # bytes in ascii umwandeln
23

Shell <
>>> %Run cat_pc_basic.py
>>> cat('fa;')
'fa007030160;'
```

24.9.26 DK2JK 16/25

Bei YAESU sieht ein CAT- Befehl wesentlich einfacher aus als bei ICOM, da die Befehle ASCII-codiert sind.

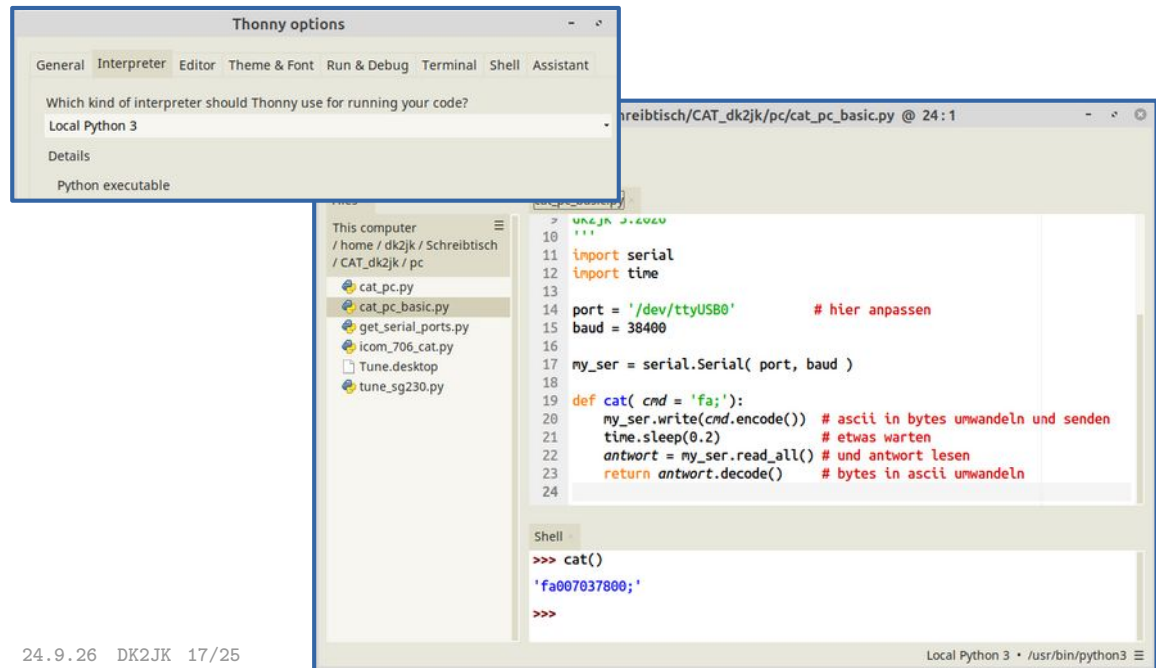
Dazu habe ich ein Python-Skript geschrieben mit der Funktion `cat()`. Übergabewert in der Klammer ist das CAT- Kommando.

Im Beispiel wird die Frequenz mit dem Befehl `,fa;'` (Frequenz VFO A) abgefragt.

Übrigens: CAT-Befehle dürfen grundsätzlich groß oder klein geschrieben werden.

Diese Funktion würde auch bei Elecraft K3, KX3 oder Kenwood funktionieren.

Experiment live



Jetzt stelle ich eine Verbindung mit dem FT-710 her.

Dazu wird Thonny mit der Option - Interpreter - Local gestartet.

Datei: /home/dk2jk/Schreibtisch/CAT_dk2jk/pc/cat_pc_basic.py (Links im Dateimanager).

Im Skript wird die Funktion `cat()` nur definiert, also nicht aufgerufen.

Die Funktion `cat()` wird jetzt interaktiv von der Kommandozeile mit verschiedenen Befehlen gestartet. --> Demo mit Kamera !

`cat()` --> `'fa007037800;'` --> Frequenz ist...

`cat('pc;')` --> `'pc044;'` --> Power ist 44 Watt

`cat('fb007150000;')` --> VF0 B auf ...

`cat('sv;')` --> swap VF0 A/B

2.1. Tune Programm am PC

Aufgabe: eine Kommandofolge senden, die den manuellen Tune-Vorgang nachbildet

```
def tune_function():  
    meter = get('MS;') # 'Meter'- Einstellung lesen  
    mode = get('MD0;') # 'Mode' lesen  
    power = get('PC;') # 'Power' lesen  
    afgain= get('AG0;') # 'AF GAIN' lesen  
    beep = get('EX030101;') # 'beep' lesen  
    write('EX030101050;') # beep auf laut , falls aus  
    write('AG0000;') # lautstärke aus  
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen  
    write('PC010;') # 'Power' auf 10 Watt schalten  
    write('MD06;') # 'Mode' auf RTTY schalten  
    write('MX1;') # 'PTT' einschalten  
    sleep(tune_dauer) # x Sekunden lang Träger senden  
    write('MX0;') # 'PTT' ausschalten  
    write( mode ) # die gelesenen Werte wieder herstellen  
    write( power )  
    write( meter )  
    write( afgain )  
    write ( beep )  
    close() # Schnittstelle schliessen  
    return
```

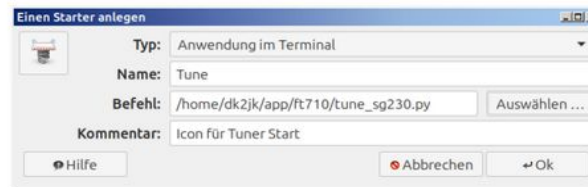
24.9.26 DK2JK 18/25

Hier sind alle Befehle zusammengefasst für einen Tune-Button, der den manuellen Tune- Vorgang nachbildet. Es werden erst alle Werte gelesen und gespeichert, die verändert werden sollen. Dann wird die Befehlsfolge zum Tunen gesendet; anschliessend werden die gespeicherten Werte wieder zurückgeschrieben.

Tune Icon (Linux)

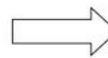
Link auf dem Linux Schreibtisch erstellen

Wählen Sie auf dem Schreibtisch mit der rechten Maustaste „Starter anlegen“ und tragen Sie unter ‚Befehl‘ den kompletten Pfad des Python-Programms ein.



Damit das Icon des „Starters“ besser aussieht, kann man im Starter-Text mit einem Texteditor folgende Zeile ergänzen: „Icon = python3“.

```
Tune.desktop *
1  #!/usr/bin/env xdg-open
2  [Desktop Entry]
3  Version=1.0
4  Type=Application
5  Terminal=true
6  Name[de_DE]=Tune
7  Icon=python3
8  Comment[de_DE]=Icon für Tuner Start
9  Exec=/home/dk2jk/app/ft710/tune_sg230.py
10 Name=Tune
11 Comment=Icon für Tuner Start
12
```



24.9.26 DK2JK 19/25

Das Tune-Programm soll über ein Icon aufgerufen werden.

Bei Linux wird dazu ein Starter-Icon angelegt. Bei „Exec=“ wird das eingetragen, was man auch im Terminal zum Start des Python-Scripts eintippen würde:

z.B.:

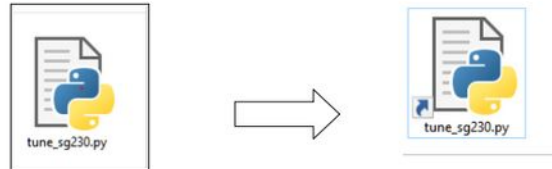
Exec=python3 /home/dk2jk/app/FT-710/tune_sg230.py

Das Icon kann man noch mit einem individuellen Symbol gestalten.

Tune Icon Link (Windows)

Link auf dem Windows Desktop erstellen

Wähle im Dateimanager das Programm aus und erstelle mit der rechten Maustaste eine Verknüpfung:

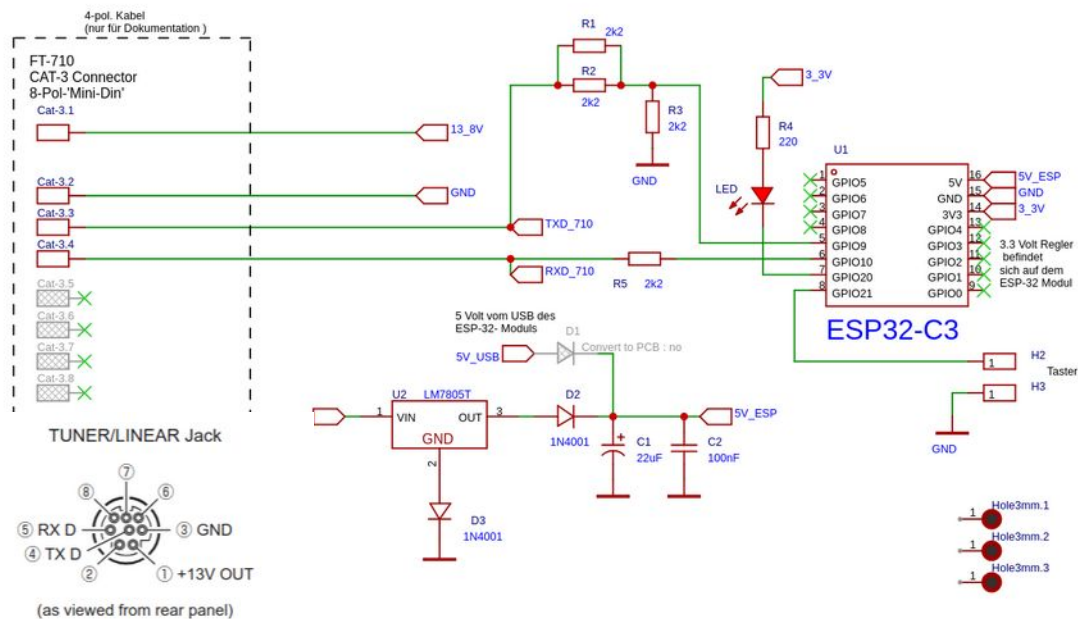


Die Verknüpfung wird als „Tuner-Start-Button“ auf den Desktop gezogen.

Beim Start über den Desktop öffnet sich ein Terminal- Fenster mit Statusanzeigen..

Bei Windows wird im Dateimanager aus dem Python-Skript-Icon ein Verknüpfungs-Icon gemacht. Dieses Icon wird dann auf den Desktop verschoben. Das Python-Skript muss die Eigenschaft „ausführbar“ haben.

2.2.Tune Taste mit ESP-32



24.9.26 DK2JK 21/25

In diesem Beispiel wird statt PC ein Mikrocontroller verwendet. Ein Taster löst die gleiche Befehlsfolge aus, wie sie beim PC-Programm beschrieben wurde; hier wird nicht von einem Icon gestartet, sondern über einen realen Taster am Mikroprozessor.

Für die Prozessor-Hardware stehen 13,8V als Stromversorgung an einem 9-pol. Mini-DIN-Stecker am Transceiver zur Verfügung. Aus den 13,8V werden über einen Spannungsregler 5V gemacht; der ESP32-Modul hat noch einen 3.3V Regler. Als CAT-Schnittstelle wird eine zweite ser. Schnittstelle des Prozessors verwendet.

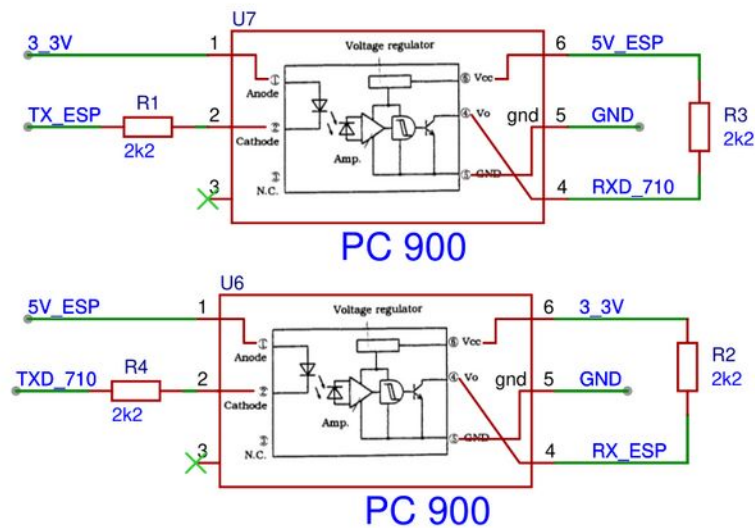
Da hier der Prozessor mit 3.3 Volt arbeitet, muss für RXD des Prozessors ein Spannungsteiler 5V/ 3,3V vorgeschaltet werden. Der Sender TXD des Prozessors ist ok, da 3,3V als High-Pegel am RXD Eingang des Transceivers als TTL-HIGH akzeptiert wird.

Die Tune-Taste ist an GPIO21 angeschlossen (rechts im Schaltbild). Taster ist aktiv-Low; ein Pullup-Widerstand ist in der CPU.

Die USB -Schnittstelle wird als virtuelle ser. Schnittstelle zum Programmieren benötigt.

Als Anmerkung: Python läuft hier im Mikroprozessor als „Micro-Python“. Für viele 32-bit-Prozessoren gibt es ein Micropython- Image . Die Programmier-Kommunikation geht über USB als virtuelle ser.

Variante mit Optokoppler



24.9.26 DK2JK 22/25

Der Vollständigkeit halber wird hier noch eine Variante des 3,3V/ 5V Pegelwandlers mit Optokoppler gezeigt.

Auf der Sendeseite wird die LED des Optokopplers getrieben.

Auf der Empfangsseite wird der Treiber des Optokopplers mit der passenden Spannung versorgt; am Pullup-Widerstand liegt der gewünschte Pegel an.

MicroPython Programm ESP32

```
def tune_function():  
    meter = get('MS;') # 'Meter'- Einstellung lesen  
    mode = get('MD0;') # 'Mode' lesen  
    power = get('PC;') # 'Power' lesen  
    afgain= get('AG0;') # 'AF GAIN' lesen  
    beep = get('EX030101;') # 'beep' lesen  
    write('EX030101050;') # beep auf laut , falls aus  
    write('AG0000;') # lautstärke aus  
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen  
    write('PC010;') # 'Power' auf 10 Watt schalten  
    write('MD06;') # 'Mode' auf RTTY schalten  
    write('MX1;') # 'PTT' einschalten  
    sleep(tune_dauer) # x Sekunden lang Träger senden  
    write('MX0;') # 'PTT' ausschalten  
    write( mode ) # die gelesenen Werte wieder herstellen  
    write( power )  
    write( meter )  
    write( afgain )  
    write( beep )  
    close()  
    return
```

```
main.py *  
#main.py  
import esp32_tune
```

```
esp32_tune.py  
while True:  
    x= taste() # taste lesen , low aktiv  
    led(x) # led an solange taste gedrueckt ist  
    if x ==0: # taste gedrueckt ?  
        tune_function()
```

24.9.26 DK2JK 23/25

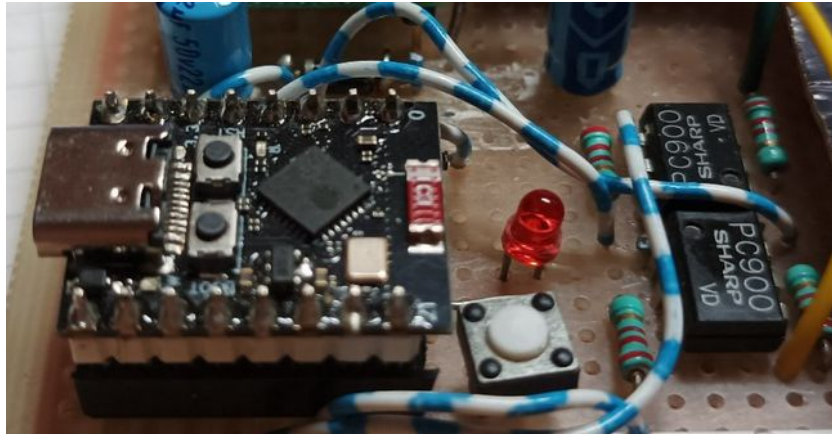
Hier sind nochmals alle Befehle zusammengefasst für einen Tune- Button. Die Funktion selbst ist identisch mit der PC-Version.

Nur die Funktionen `get()` und `write()` sehen für Micropython etwas anders aus.

Der Tune- Button (Taste) wird in einer Schleife dauernd abgefragt. Eine optische Kontrolle ist über LED vorgesehen. Wenn die Taste gedrückt wird, so wird die `Tune_funktion` gestartet.

Der Autostart des Python-Skripts wird erreicht, indem in `,main.py'` das Tune- Programm gestartet wird , siehe Ausschnitt oben rechts.

Bild „Tune Button mit ESP32“



24.9.26 DK2JK 24/25

So sieht die Tune-Button- Hardware aus; hier noch mit Optokoppler. Im „ESP32-C3-Super-Mini-Modul“ läuft das Python-Skript.

Das Ganze ist in einem Gehäuse untergebracht, wo weitere Steuerfunktionen des Tuners untergebracht sind (Reset, Lock).

Verweise

Alle Unterlagen auf Github:

<https://github.com/dk2jk/CAT--Control>

Python Download für verschiedene Betriebssysteme:

<https://www.python.org/downloads/>

Thonny Python- Entwicklungs- Oberfläche (IDE):

<https://thonny.org/>

MicroPython Download für Microcontroller:

<https://micropython.org/download/>

ICOM CI-V REFERENCE GUIDE (English):

<https://www.icomjapan.com/support/manual/3064/>

YAESU CAT Reference:

https://www.radiomanual.info/schemi/YAESU_HF/FT-710_CAT_user.pdf

Elecraft KXPA100 Amplifier Command Reference:

<https://ftp.elecraft.com/KXPA>